

Date:.....

1. TITLE OF THE LAB EXPERIMENT:

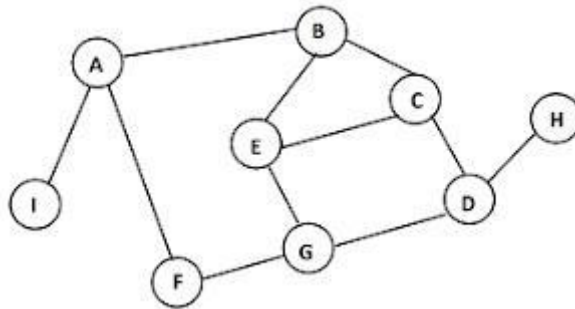
Implement BFS and DFS demonstration and Code.

2. OBJECTIVES:

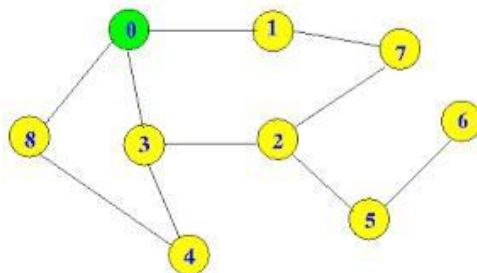
- The BFS and DFS algorithms for traversing a graph.
- BFS and DFS are fundamental algorithms in graph theory and are used for various tasks such as finding connected components, detecting cycles, and finding paths between vertices.

3. PROCEDURE:

BFS: BFS specifically can be used to find the shortest path between two nodes in an unweighted graph. Breadth-First Search can be used as a traversal method to find all the neighboring nodes in a Peer-to-Peer Network. For example, BitTorrent uses Breadth-First Search for peer-to-peer communication. so that was all about the working of the Breadth - First Search algorithm.



DFS: DFS is often used as a subroutine in more complex algorithms. The outcome of a DFS traversal of a graph is a spanning. A spanning tree is a graph that is devoid of loops. To implement DFS traversal, you need to utilize a stack data structure with a maximum size equal to the total number of vertices in the graph.



4. IMPLEMENTATION: BFS (Source Code):

```
+ Code + Text
graph = {
    'A' : ['B','F','I'],
    'B' : ['A','E','C'],
    'C' : ['E','D'],
    'D' : ['G','H'],
    'E' : ['B','C','G'],
    'F' : ['A','G'],
    'G' : ['D','E','F'],
    'H' : ['D'],
    'I' : ['A']
}

visited = []
queue = []

def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)

    while queue:
        m = queue.pop(0)
        print (m, end = " ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')    # function calling
```

DFS (Source Code):

```
+ Code + Text
# Using a Python dictionary to act as an adjacency list
graph = {
    '0' : ['1','3','8'],
    '1' : ['0','7'],
    '2' : ['3','5','7'],
    '3' : ['0','2','4'],
    '4' : ['3','8'],
    '5' : ['2','6'],
    '6' : ['5'],
    '7' : ['1','2'],
    '8' : ['0','4']
}

visited = set()

def dfs(visited, graph, node):
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Depth-First Search")
dfs(visited, graph, '0')
```

5. OUTPUT: BFS (output)

```
# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')    # function calling
```

```
⇒ Following is the Breadth-First Search
A B F I E C G D H
```

DFS (output):

```
⇒ Following is the Depth-First Search
0
1
7
2
3
4
8
5
6
```

6. DISCUSSION:

After completing this experiment, BFS explores all the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level. DFS explores as far as possible along each branch before backtracking. BFS guarantees to find the shortest path from the starting node to any other reachable node in an unweighted graph. DFS is not guaranteed to find the shortest path from the starting node to any other reachable node, but it's useful for tasks like topological sorting, finding strongly connected components, etc. each with its own advantages and use cases. Depending on the problem requirements, one might be preferred over the other.



Green University of Bangladesh
Department of Computer Science and Engineering
(CSE)

Faculty of Sciences and Engineering
Semester: (Spring, Year: 2023), B.Sc. in CSE (Day)

LAB REPORT NO :03

Course Title: Artificial Intelligence Lab

Course Code: CSE-316

Section: 211D2

Lab Experiment Name: Iterative Deepening Depth First Search (IDDFS)

Student Details

Name		ID
1.	Md Mahdi Hasan	211002069

Lab Date : 19-03-24

Submission Date : 31-03-24

Course Teacher's Name : Md Naimul Pathan

Lab Report Status

Marks:

Signature:.....

Comments:.....

Date:.....

1. **TITLE:** Iterative Deepening Depth First Search (IDDFS).

2. **OBJECTIVES:** There are two common ways to traverse a graph, BFS and DFS. Considering a Tree (or Graph) of huge height and width, both BFS and DFS are not very efficient due to following reasons.

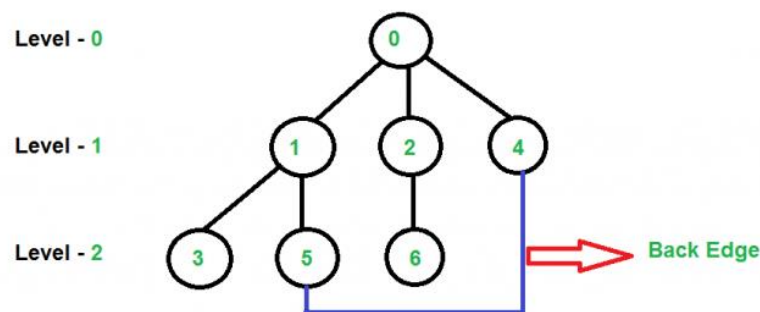
DFS: first traverses nodes going through one adjacent of root, then next adjacent. The problem with this approach is, if there is a node close to root, but not in first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find shortest path to a node

BFS: goes level by level, but requires more space. The space required by DFS is $O(d)$ where d is depth of tree, but space required by BFS is $O(n)$ where n is number of nodes in tree.

IDDFS: combines depth-first search's space-efficiency and breadth-first searches.

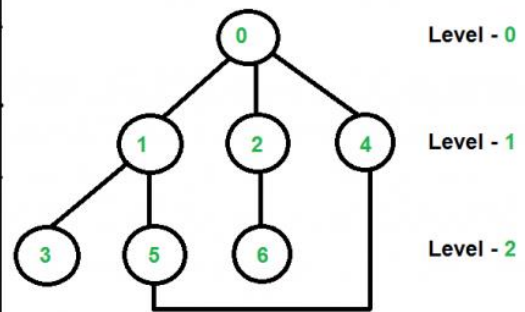
3. **PROCEDURE:** There can be two cases:

- When the graph has no cycle: This case is simple. We can DFS multiple times with different height limits.
- When the graph has cycles. This is interesting as there is no visited flag in IDDFS.



Although, at first sight, it may seem that since there are only 3 levels, so we might think that Iterative Deepening Depth First Search of level 3, 4, 5,...and so on will remain same. But, this is not the case. You can see that there is a cycle in the above graph, hence IDDFS will change for level-3,4,5..and so on.

Depth	Iterative Deepening Depth First Search
0	0
1	0 1 2 4
2	0 1 3 5 2 6 4 5
3	0 1 3 5 4 2 6 4 5 1



The explanation of the above pattern is left to the readers.

4. IMPLEMENTATION:

```

from collections import defaultdict

class Graph:

    def __init__(self, vertices):
        self.V = vertices
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def DLS(self, src, target, maxDepth):
        if src == target:
            return True
        if maxDepth <= 0:
            return False
        for i in self.graph[src]:
            print(i, end=' ')
            if self.DLS(i, target, maxDepth - 1):
                return True
        return False

    def IDDFS(self, src, target, maxDepth):
        print("Depth 0:", src, end=' ')
        for i in range(1, maxDepth + 1):
            print(f"\nDepth {i}: A ", end='')
            if self.DLS(src, target, i):
                return True
        return False

```

```

g = Graph(10)
g.addEdge('A', 'B')
g.addEdge('A', 'C')
g.addEdge('B', 'D')
g.addEdge('B', 'E')
g.addEdge('C', 'F')
g.addEdge('C', 'G')
g.addEdge('D', 'H')
g.addEdge('D', 'I')
g.addEdge('F', 'K')

target = 'G'
maxDepth = 3
src = 'A'

if g.IDDFS(src, target, maxDepth):
    print("\nTarget is reachable from source within max depth")
else:
    print("\nTarget is NOT reachable from source within max depth")

```

5. OUTPUT:

```

Depth 0: A
Depth 1: A B C
Depth 2: A B D E C F G
Target is reachable from source within max depth

```

6. ANALYSIS AND DISCUSSION: Iterative Deepening Depth First Search (IDDFS) efficiently explores large search spaces with limited memory by incrementally deepening the search limit. It ensures optimality in finding the shortest path while maintaining a low memory footprint similar to DFS. However, it may struggle with high branching factors and non-uniform edge costs.



Green University of Bangladesh
Department of Computer Science and Engineering
(CSE)

Faculty of Sciences and Engineering
Semester: (Spring, Year: 2023), B.Sc. in CSE (Day)

LAB REPORT NO :04

Course Title: Artificial Intelligence Lab

Course Code: CSE-316

Section: 211D2

Lab Experiment Name: Solve 8 queen problems

Student Details

Name		ID
1.	Md Mahdi Hasan	211002069

Lab Date : 07-05-24

Submission Date : 15-05-24

Course Teacher's Name : Md Naimul Pathan

Lab Report Status

Marks:

Comments:.....

Signature:.....

Date:.....

1. TITLE: Solve 8 queen problems

2. OBJECTIVES: N - Queens problem is to place n - queens in such a manner on an n x n chessboard that no queens attack each other by being in the same row, column or diagonal. The eight queens' problem is the problem of placing eight queens on an 8x8 chessboard such that none of them attack one another (no two are in the same row, column, or diagonal).

3. PROCEDURE:

Problem Statement

- Place eight queens on an 8x8 chessboard.
- No queen should be able to attack any other queen.
- A queen can attack any piece in the same row, column, or diagonal.

Backtracking Approach

1. Start in the leftmost column.
2. If all queens are placed, then return true.
3. Try all rows in the current column. For each row, if the queen can be placed safely in this row, then mark this cell and try to solve the problem recursively for the rest of the columns.
4. If placing the queen in the current row and proceeding to solve the rest of the problem leads to a solution, then return true.
5. If placing a queen in any row in the current column and proceeding with this placement does not lead to a solution, then backtrack and return false.
6. If all rows have been tried and none worked, return false to trigger backtracking.

4. IMPLEMENTATION:

```
main.py x +
main.py > f solve_nq_util > ...

1  N = 8
2
3  board = [[0] * N for _ in range(N)]
4
5  def print_solution(board):
6      for row in board:
7          print(' '.join('Q' if x else '.' for x in row))
8
9  def is_safe(board, row, col):
10
11      for i in range(col):
12          if board[row][i] == 1:
13              return False
14
15
16      for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
17          if board[i][j] == 1:
18              return False
19
20
21      for i, j in zip(range(row, N, 1), range(col, -1, -1)):
22          if board[i][j] == 1:
23              return False
24
25      return True
26
```

```
def solve_nq_util(board, col):
    if col >= N:
        return True

    for i in range(N):
        if is_safe(board, i, col):
            board[i][col] = 1

            if solve_nq_util(board, col + 1) == True:
                return True

            board[i][col] = 0

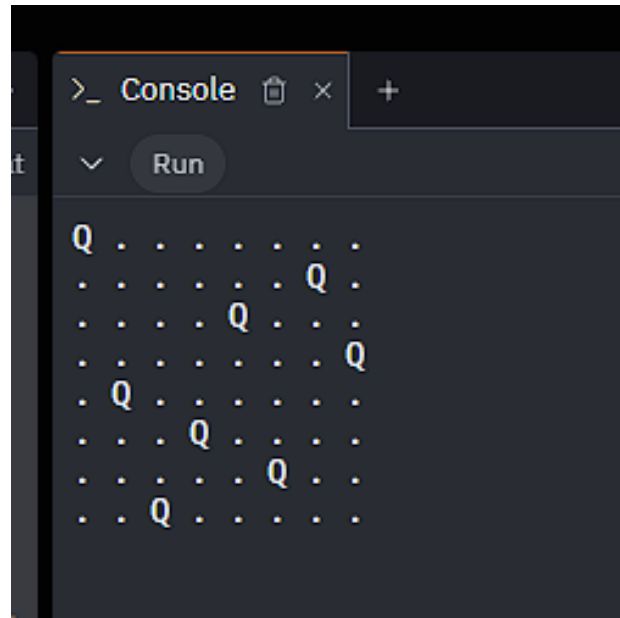
    return False

def solve_nq():
    if not solve_nq_util(board, 0):
        print("Solution does not exist")
        return False

    print_solution(board)
    return True

solve_nq()
```

5. OUTPUT:



```
>_ Console [icon] x +
  v Run
Q . . . . . .
. . . . . Q .
. . . . Q . .
. . . . . Q
. Q . . . . .
. . . Q . . .
. . . . Q . .
. . Q . . . .
```

6. ANALYSIS AND DISCUSSION: The 8 Queens Problem is a quintessential example in computer science, illustrating the profound concept of backtracking. Studying this problem is crucial as it fosters a deep understanding of algorithmic design and enhances problem-solving skills, pivotal for tackling complex computational problems.



Green University of Bangladesh
Department of Computer Science and Engineering
(CSE)

Faculty of Sciences and Engineering
Semester: (Spring, Year: 2023), B.Sc. in CSE (Day)

LAB REPORT NO :04

Course Title: Artificial Intelligence Lab

Course Code: CSE-316

Section: 211D2

Lab Experiment Name: K-Means Clustering Algorithm

Student Details

Name		ID
1.	Md Mahdi Hasan	211002069

Lab Date : 31-05-24

Submission Date : 04-06-24

Course Teacher's Name : Md Naimul Pathan

Lab Report Status

Marks:

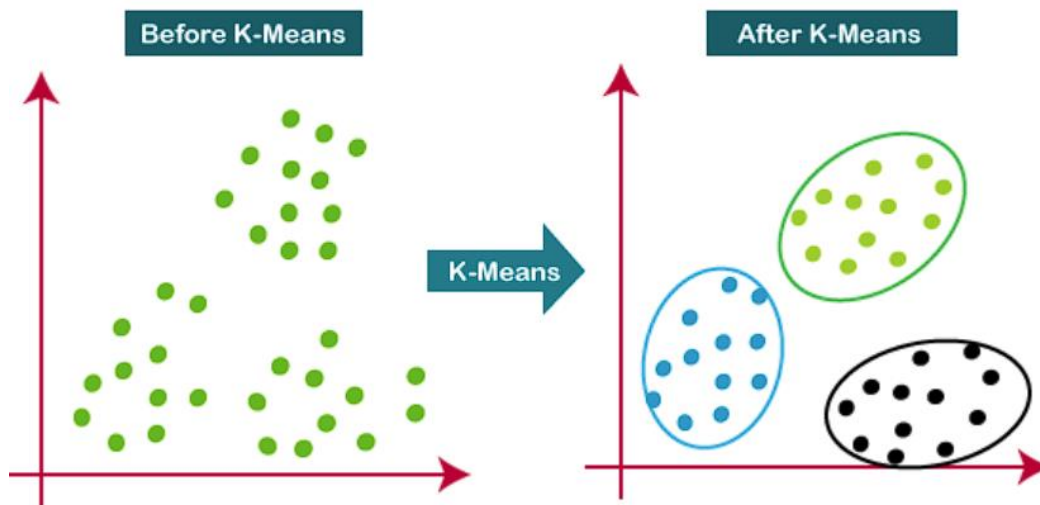
Comments:.....

Signature:.....

Date:.....

1. **TITLE:** K-Means Clustering Algorithm

2. **OBJECTIVES:** K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems in machine learning. The k-means clustering algorithm mainly performs two tasks:



The K-Means Algorithm Work:

The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.

Elbow Method:

$$WCSS = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i C_2)^2 + \sum_{P_i \text{ in Cluster3}} \text{distance}(P_i C_3)^2$$

3. PROCEDURE:

Data Preparation	Perform any necessary data preprocessing steps such as handling missing values, feature scaling, and encoding categorical variables.
Used and alternative resources	Split both datasets into training and testing sets. Train various classification algorithms (K-Means Algorithms) on the training set of each dataset.
Unsupervised Learning (Clustering)	Apply the K-means clustering algorithm to both datasets. Visualize the clustering results using scatter plots, focusing on features such as Sepal Length vs Petal Width. Interpret the clustering results and assess their coherence and relevance.
Comparison and Analysis	Analyses any differences in model effectiveness, feature importance, clustering results, and scalability between the datasets.

4. IMPLEMENTATION: (Load The DataSet:)

<code>iris= datasets.load_iris()</code>
<code>x=pd.DataFrame(iris.data, columns=['SL','SW','PL','PW'])</code>
<code>y=pd.DataFrame(iris.target, columns=['Target'])</code>

Unsupervised Learning (Clustering):

<code>import matplotlib.patches as mpatche plt.figure(figsize=(12,3)) colors = np.array(['red','green','blue'])</code>
<code>iris_targets_legend = np.array(iris.target_names) red_patch = mpatches.Patch(color='red',label='Setosa')</code>
<code>green_patch = mpatches.Patch(color='green',label='Versicolor') blue_patch =</code>
<code>mpatches.Patch(color='blue',label='Verginica')</code>

Iris k-mean model cluster Model:

<code>iris_k_mean_model = KMeans(n_clusters=3,n_init=10) iris_k_mean_model.fit(x)</code>
<code>print(iris_k_mean_model.predict(x)) print(iris_k_mean_model.cluster_centers_)</code>

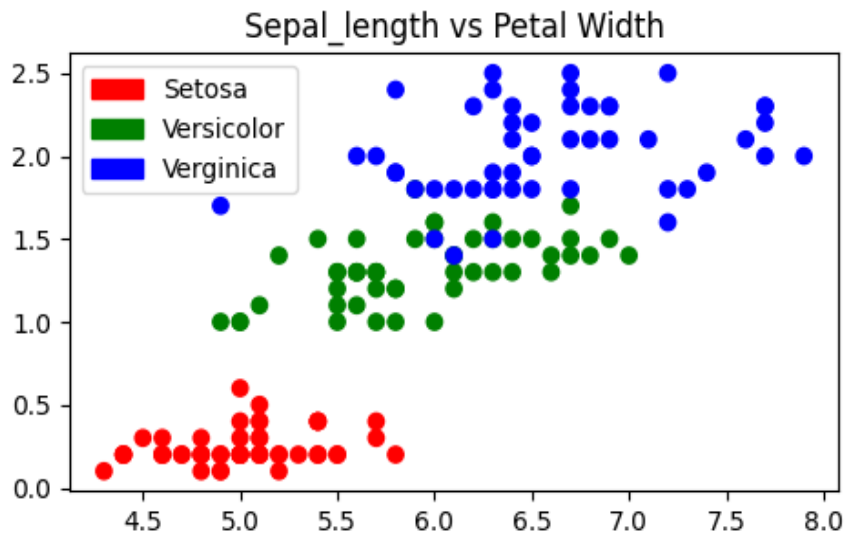
Find accuracy of the trained model:

<code>plt.subplot(1,2,2) plt.scatter(x['SL'], x['PW'], c=colors[predictedY]) plt.title("Models classification")</code>
<code>plt.legend(handles=[red_patch, green_patch, blue_patch]) print("Accuracy Score:"+</code>
<code>str(ss.accuracy_score(predictedY,y['Target'])))</code>

5. RESULT:

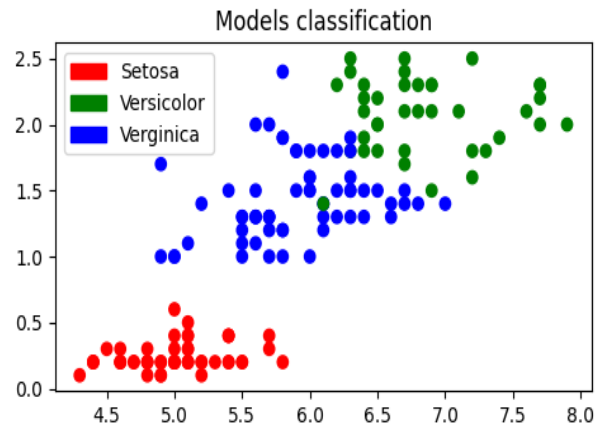
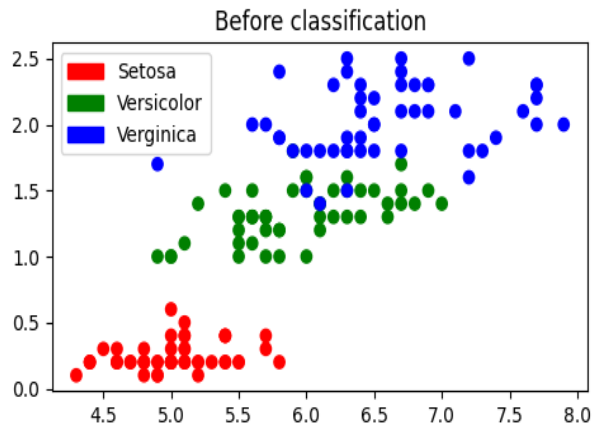
Iris k-mean model cluster Model

Visualize the data using scatter plots:

[illegible]

Check the classification performance of the trained model

Check the Accuracy and Array:



Accuracy Score:0.44

```
array([[50,  0,  0],  
       [ 0,  2, 36],  
       [ 0, 48, 14]])
```

6. ANALYSIS AND DISCUSSION: In conclusion, K-means clustering is a powerful unsupervised machine learning algorithm for grouping unlabeled datasets. Its objective is to divide data into clusters, making similar data points part of the same group. The algorithm initializes cluster centroids and iteratively assigns data points to the nearest centroid, updating centroids based on the mean of points in each cluster.